6. SERVICE VALIDATION

Each service is checked against the five properties of a service. **Yes** means the property holds as designed; **Partial** means it holds only under assumptions we have not yet guaranteed; **No** means the current design fails it. Every entry that is not **Yes** is explained in 6.2 with the fix we would apply.

6.1 Validation Matrix

Service	Reachable	Self-contained	Has a contract	Independent	Loosely coupled
User Service	Yes	Yes	Yes	Yes	Yes
Catalogue Service	Yes	Yes	Yes	Yes	Partial
Order Service	Yes	Yes	Yes	No	Partial
Payment Service	Partial	Yes	Yes	Yes	Yes

6.2 Gaps and Fixes

Service	Property	Status	Why it fails	How we would fix it
Order Service	Independent	No	**`placeOrder`** calls User, Catalogue and Payment synchronously. If any one of them is down, no order can be placed at all, so the service cannot operate on its own.	Create the order in **`PENDING`** state first and confirm it when the payment result arrives, so an order is never lost after a successful charge. Add timeouts and retries on every outbound call, and fail the request with a clear error instead of hanging.
Order Service	Loosely coupled	Partial	`order_items.food_item_ref` points at Catalogue rows. If Catalogue deletes an item or reuses an ID, past orders become wrong or unreadable.	Catalogue soft-deletes items (`is_available = FALSE`) and never reuses an ID. Order Service already stores `item_name` and `unit_price` as a snapshot, so order history renders correctly even if the item later changes or disappears.
Catalogue Service	Loosely coupled	Partial	**`manageMenu`** and **`manageFoodService`** are restricted operations, so the service currently has to ask User Service to check the caller on every request.	Issue signed tokens from User Service and have Catalogue verify the signature locally with a public key. Catalogue then authorises callers without a network call, removing the runtime dependency.
Payment Service	Reachable	Partial	Only Order Service currently calls it. **`checkPaymentStatus`** has no route exposed to clients, so a user cannot check a payment they started.	Publish **`checkPaymentStatus`** through the API gateway on a stable address, with the caller's identity checked against the order that owns the payment.

User Service passes all five properties: it is reachable at a published address, owns its data end to end, exposes a defined contract, calls no other service, and is referenced by others only through opaque user identifiers.